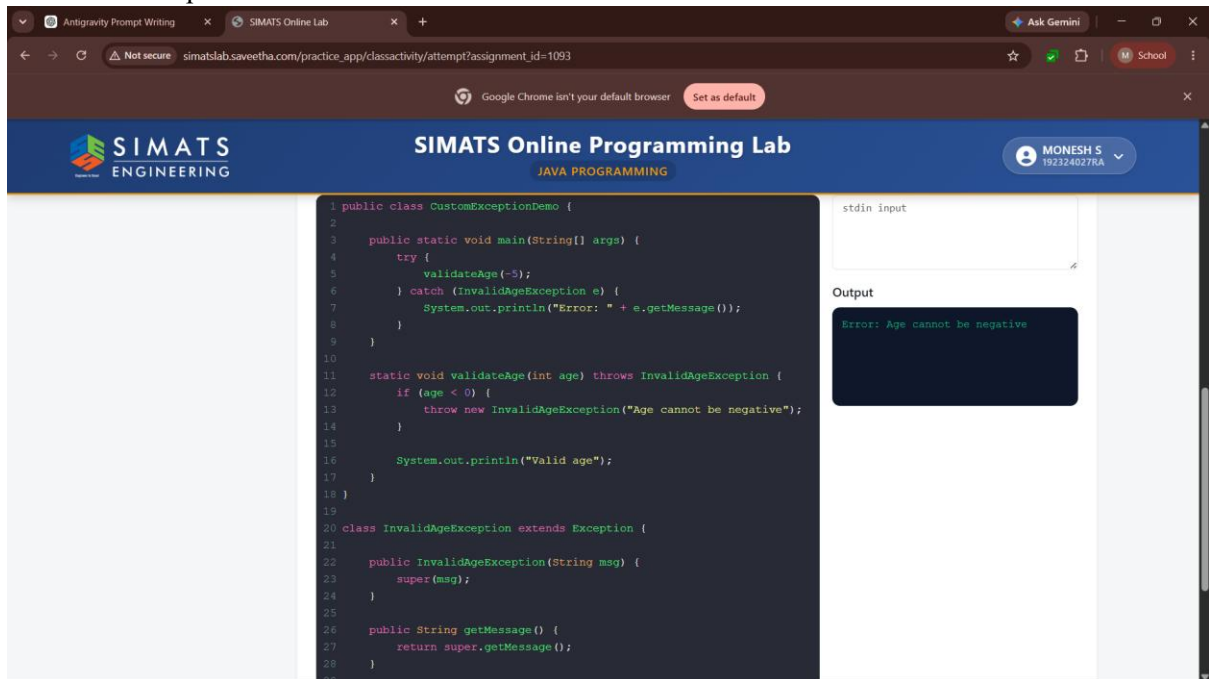


CO3-AT1-Debugging

1. Custom Exception Class

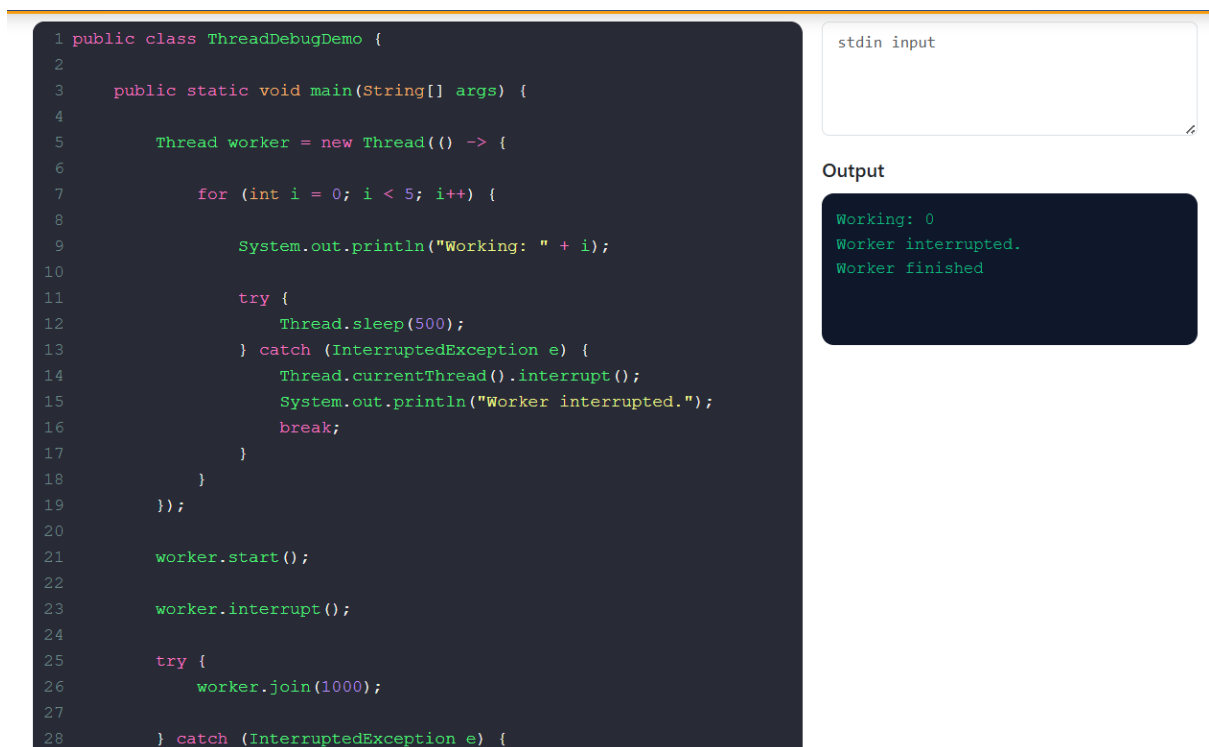


The screenshot shows the SIMATS Online Programming Lab interface. The browser address bar displays "simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=1093". The lab header includes the SIMATS ENGINEERING logo, the title "SIMATS Online Programming Lab", and the user "MONESH S 192324027RA". The main area contains a code editor with the following Java code:

```
1 public class CustomExceptionDemo {
2
3     public static void main(String[] args) {
4         try {
5             validateAge(-5);
6         } catch (InvalidAgeException e) {
7             System.out.println("Error: " + e.getMessage());
8         }
9     }
10
11     static void validateAge(int age) throws InvalidAgeException {
12         if (age < 0) {
13             throw new InvalidAgeException("Age cannot be negative");
14         }
15
16         System.out.println("Valid age");
17     }
18 }
19
20 class InvalidAgeException extends Exception {
21
22     public InvalidAgeException(String msg) {
23         super(msg);
24     }
25
26     public String getMessage() {
27         return super.getMessage();
28     }
29 }
```

On the right, there is a "stdin input" field and an "Output" box. The output box displays the message: "Error: Age cannot be negative".

2. Thread join and Interruption



The screenshot shows the SIMATS Online Programming Lab interface. The browser address bar displays "simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=1093". The lab header includes the SIMATS ENGINEERING logo, the title "SIMATS Online Programming Lab", and the user "MONESH S 192324027RA". The main area contains a code editor with the following Java code:

```
1 public class ThreadDebugDemo {
2
3     public static void main(String[] args) {
4
5         Thread worker = new Thread(() -> {
6
7             for (int i = 0; i < 5; i++) {
8
9                 System.out.println("Working: " + i);
10
11                 try {
12                     Thread.sleep(500);
13                 } catch (InterruptedException e) {
14                     Thread.currentThread().interrupt();
15                     System.out.println("Worker interrupted.");
16                     break;
17                 }
18             }
19         });
20
21         worker.start();
22
23         worker.interrupt();
24
25         try {
26             worker.join(1000);
27         } catch (InterruptedException e) {
28
29 }
```

On the right, there is a "stdin input" field and an "Output" box. The output box displays the following messages:

```
Working: 0
Worker interrupted.
Worker finished
```

3. Wait() and Notify() Coordination

```
1 public class WaitNotifyDemo {
2
3     private boolean dataReady = false;
4     private final Object lock = new Object();
5
6     public void produce() {
7         synchronized (lock) {
8             dataReady = true;
9             System.out.println("Data produced");
10            lock.notify();
11        }
12    }
13
14    public void consume() {
15        synchronized (lock) {
16            if (!dataReady) {
17                try {
18                    lock.wait();
19                } catch (InterruptedException e) {
20                    Thread.currentThread().interrupt();
21                }
22            }
23
24            System.out.println("Data consumed");
25        }
26    }
27
28    public void consumeCorrectly() {
29        synchronized (lock) {
```

stdin input

Output

```
Data produced
Data consumed
```

4. Multiple Catch Blocks Ordering

```
1 public class MultiCatchDemo {
2
3     public static void main(String[] args) {
4
5         try {
6             processData(args);
7         } catch (ArithmeticException e) {
8             System.out.println("Math error");
9         } catch (NumberFormatException e) {
10            System.out.println("Format error");
11        } catch (Exception e) {
12            System.out.println("General error: " + e.getMessage());
13        }
14
15        try {
16            int x = 10 / 0;
17        } catch (ArithmeticException e) {
18            System.out.println(e);
19        }
20
21        try {
22            Object o = "text";
23            Integer i = (Integer) o;
24        } catch (ClassCastException e) {
25            System.out.println("Cast failed");
26        }
27    }
28
29    static void processData(String[] args) {
```

stdin input

Output

```
General error: Index 0 out of
bounds for length 0
java.lang.ArithmeticException: /
by zero
Cast failed
```